

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА
ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №1

“Інтерполяція кубічними сплайнами”

Виконав:

Студент 3 курсу

групи ФЕП-33

Абдулханов Абдул-Рахім

Викладач:

ас. Каськун О. Д.

Львів-2026

Мета роботи: вивчити метод інтерполяції кубічними сплайнами, побудувати модель висотного профілю та розрахувати фізичні і градієнтні характеристики географічного маршруту Заросляк - Говерла.

Теоретичні відомості:

Кубічний сплайн є математичною моделлю тонкого, пружного і гнучкого стержня, який закріплено у заданих опорних вузлах. Стержень деформується так, щоб мінімізувати потенціальну енергію згину, що описується диференціальним рівнянням $S^{(4)}(x) = 0$. Інтерполянт є кусково-поліноміальною функцією третього степеня, двічі неперервно диференційовною на всьому відрізку (належить класу C^2).

На кожному елементарному сегменті $[x_{i-1}, x_i]$ рівняння сплайна записується у вигляді:

$$S_i(x) = a_i + b_i(x - x_{i-1}) + c_i(x - x_{i-1})^2 + d_i(x - x_{i-1})^3, \text{ де } i = 1, \dots, n.$$

Для знаходження $4n$ невідомих коефіцієнтів формується замкнена система з $4n$ рівнянь:

1. Умови проходження через вузли: $S_i(x_{i-1}) = y_{i-1}$ та $S_i(x_i) = y_i$ ($2n$ рівнянь);
2. Умови гладкості (неперервність 1-ї та 2-ї похідних у внутрішніх вузлах): $S'_i(x_i) = S'_{i+1}(x_i)$ та $S''_i(x_i) = S''_{i+1}(x_i)$ ($2n - 2$ рівняння);
3. Крайові умови вільного (природного) сплайна на межах: $S''_1(x_0) = 0$ та $S''_n(x_n) = 0$ (2 рівняння).

Виразивши коефіцієнти a_i, b_i, d_i через другі похідні c_i , вихідна система зводиться до тридіагональної СЛАУ n -го порядку відносно c_i :

$$h_{i-1}c_{i-1} + 2(h_{i-1} + h_i)c_i + h_ic_{i+1} = 3((y_i - y_{i-1})/h_i - (y_{i-1} - y_{i-2})/h_{i-1}), \quad i = 2, \dots, n-1,$$

$$\text{при } c_1 = 0 \text{ та } h_{n-1}c_{n-1} + 2(h_{n-1} + h_n)c_n = 3((y_n - y_{n-1})/h_n - (y_{n-1} - y_{n-2})/h_{n-1}).$$

Отримана система розв'язується прямим методом прогонки (алгоритм Томаса), що потребує лише $O(n)$ арифметичних операцій. Матриця системи має строге діагональне переважання: $|\beta_i| = 2(h_{i-1} + h_i) > h_{i-1} + h_i = |\alpha_i| + |\gamma_i|$, що гарантує абсолютну стійкість методу до накопичення похибок округлення та відсутність ділення на нуль.

Виконання роботи

1. Зчитав дані GPS-координат та висот 21 точки маршруту урочище Заросляк - вершина гори Говерла. Використав формулу гаверсинуса для обчислення сферичної кумулятивної відстані між послідовними координатами в метрах:

```
1 import os
2 import json
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 # гаверсинус для обчислення відстані між двома координатами
7 def haversine(lat1, lon1, lat2, lon2):
8     r = 6371000 # радіус землі в метрах
9     phi1, phi2 = np.radians(lat1), np.radians(lat2)
10    dphi = np.radians(lat2 - lat1)
11    dlamba = np.radians(lon2 - lon1)
12    a = np.sin(dphi / 2.0)**2 + np.cos(phi1) * np.cos(phi2) * np.sin(dlamba / 2.0)**2
13    return 2.0 * r * np.arctan2(np.sqrt(a), np.sqrt(1.0 - a))
14
15 # метод прогонки для розв'язку трьохдіагональної спарсної слау
16 def solve_tridiagonal(alpha, beta, gamma, delta):
17     n = len(beta)
18     cap_a = np.zeros(n)
19     cap_b = np.zeros(n)
20
21     # пряма прогонка
22     cap_a[0] = -gamma[0] / beta[0]
23     cap_b[0] = delta[0] / beta[0]
24
25     for i in range(1, n - 1):
26         denom = alpha[i] * cap_a[i - 1] + beta[i]
27         cap_a[i] = -gamma[i] / denom
28         cap_b[i] = (delta[i] - alpha[i] * cap_b[i - 1]) / denom
29
30     denom_last = alpha[n - 1] * cap_a[n - 2] + beta[n - 1]
31
32     # зворотна прогонка
33     x = np.zeros(n)
34     x[n - 1] = (delta[n - 1] - alpha[n - 1] * cap_b[n - 2]) / denom_last
35
36     for i in range(n - 2, -1, -1):
37         x[i] = cap_a[i] * x[i + 1] + cap_b[i]
```

Результати табуляції вхідних GPS-точок:

Кількість вузлів: 21

Табуляція вузлів:

№	Latitude	Longitude	Elevation (m)
0	48.164214	24.536044	1264.00
1	48.164983	24.534836	1285.00
2	48.165605	24.534068	1285.00
3	48.166228	24.532915	1333.00
4	48.166777	24.531927	1310.00
5	48.167326	24.530884	1318.00
6	48.167011	24.530061	1318.00
7	48.166053	24.528039	1339.00
8	48.166655	24.526064	1375.00
9	48.166497	24.523574	1417.00
10	48.166128	24.520214	1486.00
11	48.165416	24.517170	1524.00
12	48.164546	24.514640	1553.00
13	48.163412	24.512980	1630.00
14	48.162331	24.511715	1757.00
15	48.162015	24.509462	1794.00
16	48.162147	24.506932	1828.00
17	48.161751	24.504244	1887.00
18	48.161197	24.501793	1975.00
19	48.160580	24.500537	1975.00
20	48.160250	24.500106	2031.00

Результати розрахунку кумулятивної відстані уздовж маршруту (x_i) та висот (y_i):

Табуляція (відстань, висота):		
№	Distance (m)	Elevation (m)
0	0.00	1264.00
1	123.85	1285.00
2	213.45	1285.00
3	323.50	1333.00
4	418.87	1310.00
5	517.41	1318.00
6	587.78	1318.00
7	771.72	1339.00
8	932.77	1375.00
9	1118.27	1417.00
10	1370.81	1486.00
11	1610.05	1524.00
12	1821.16	1553.00
13	1997.39	1630.00
14	2149.88	1757.00
15	2320.63	1794.00
16	2508.86	1828.00
17	2713.03	1887.00
18	2904.98	1975.00
19	3020.67	1975.00
20	3069.34	2031.00

2. Побудував тридіагональну СЛАУ для відшукування других похідних сплайна s_i та реалізував метод прогонки (прямий і зворотний хід). Розрахував коефіцієнти a_i , b_i , c_i , d_i для кожного відрізка маршруту:

```

1     return x
2
3     # побудова коефіцієнтів кубічних сплайнів
4     def build_cubic_spline(x_nodes, y_nodes):
5         m = len(x_nodes)
6         n = m - 1 # кількість інтервалів
7
8         h = np.diff(x_nodes)
9
10        alpha = np.zeros(n)
11        beta = np.zeros(n)
12        gamma = np.zeros(n)
13        delta = np.zeros(n)
14
15        # перше рівняння: c1 = 0
16        beta[0] = 1.0
17        gamma[0] = 0.0
18        delta[0] = 0.0
19
20        # проміжні рівняння для c2..c_{n-1}
21        for i in range(1, n - 1):
22            alpha[i] = h[i] - 1]
23            beta[i] = 2.0 * (h[i - 1] + h[i])
24            gamma[i] = h[i]
25            delta[i] = 3.0 * ((y_nodes[i + 1] - y_nodes[i]) / h[i] - (y_nodes[i] - y_nodes[i - 1]) / h[i - 1])
26
27        # останнє рівняння для c_n
28        if n > 1:
29            alpha[n - 1] = h[n - 2]
30            beta[n - 1] = 2.0 * (h[n - 2] + h[n - 1])
31            gamma[n - 1] = 0.0
32            delta[n - 1] = 3.0 * ((y_nodes[n] - y_nodes[n - 1]) / h[n - 1] - (y_nodes[n - 1] - y_nodes[n - 2]) / h[n - 2])
33
34        # знаходимо вектор c
35        c = solve_tridiagonal(alpha, beta, gamma, delta)
36
37        # коефіцієнти для кожного відрізка
38        a = np.zeros(n)
39        b = np.zeros(n)
40        d = np.zeros(n)
41
42        for i in range(n):
43            a[i] = y_nodes[i]
44            c_curr = c[i]
45            c_next = c[i + 1] if i < n - 1 else 0.0
46            d[i] = (c_next - c_curr) / (3.0 * h[i])
47            b[i] = (y_nodes[i + 1] - y_nodes[i]) / h[i] - (h[i] / 3.0) * (c_next + 2.0 * c_curr)
48
49        return a, b, c, d, alpha, beta, gamma, delta

```

Значення коефіцієнтів тридіагональної системи (alpha, beta, gamma, delta):

Коефіцієнти трьохдіагональної системи (alpha, beta, gamma, delta):					
i	alpha	beta	gamma	delta	
1	0.00	1.00	0.00	0.000000	
2	123.85	426.90	89.60	-0.508683	
3	89.60	399.30	110.05	1.308488	
4	110.05	410.84	95.37	-2.031978	
5	95.37	387.82	98.54	0.967049	
6	98.54	337.82	70.37	-0.243559	
7	70.37	508.63	183.94	0.342497	
8	183.94	689.98	161.04	0.328125	
9	161.04	693.09	185.50	0.008620	
10	185.50	876.09	252.55	0.140411	
11	252.55	983.56	239.24	-0.343138	
12	239.24	900.69	211.11	-0.064406	
13	211.11	774.68	176.23	0.898660	
14	176.23	657.43	152.48	1.187858	
15	152.48	646.48	170.76	-1.848587	
16	170.76	717.96	188.22	-0.108129	
17	188.22	784.79	204.17	0.324998	
18	204.17	792.24	191.95	0.508478	
19	191.95	615.29	115.70	-1.375386	
20	115.70	328.73	0.00	3.452056	

Обчислені коефіцієнти кубічних сплайнів a_i , b_i , c_i , d_i для 20 інтервалів:

```

Коефіцієнти кубічних сплайнів для повної траси:
i | a_i (m) | b_i | c_i | d_i
1 | 1264.00 | 0.270269 | 0.00000000 | -0.0000065656
2 | 1285.00 | -0.031855 | -0.00243944 | 0.0000311944
3 | 1285.00 | 0.282280 | 0.00594547 | -0.0000413189
4 | 1333.00 | 0.089625 | -0.00769606 | 0.0000443282
5 | 1310.00 | -0.168760 | 0.00498680 | -0.0000248661
6 | 1318.00 | 0.089685 | -0.00236402 | 0.0000154831
7 | 1318.00 | -0.013010 | 0.00090470 | -0.0000011597
8 | 1339.00 | 0.202104 | 0.00026476 | -0.0000008174
9 | 1375.00 | 0.223778 | -0.00013017 | 0.0000007784
10 | 1417.00 | 0.255835 | 0.00030298 | -0.0000009272
11 | 1486.00 | 0.231467 | -0.00039947 | 0.0000004008
12 | 1524.00 | 0.109149 | -0.00011181 | 0.0000011629
13 | 1553.00 | 0.217416 | 0.00062466 | 0.0000035231
14 | 1630.00 | 0.765850 | 0.00248733 | -0.0000134294
15 | 1757.00 | 0.587650 | -0.00365598 | 0.0000086877
16 | 1794.00 | 0.099031 | 0.00079452 | -0.0000019177
17 | 1828.00 | 0.194302 | -0.00028836 | 0.0000036832
18 | 1887.00 | 0.537180 | 0.00196770 | -0.0000123879
19 | 1975.00 | -0.076667 | -0.00516572 | 0.0000503765
20 | 1975.00 | 0.750989 | 0.01231941 | -0.0000843795

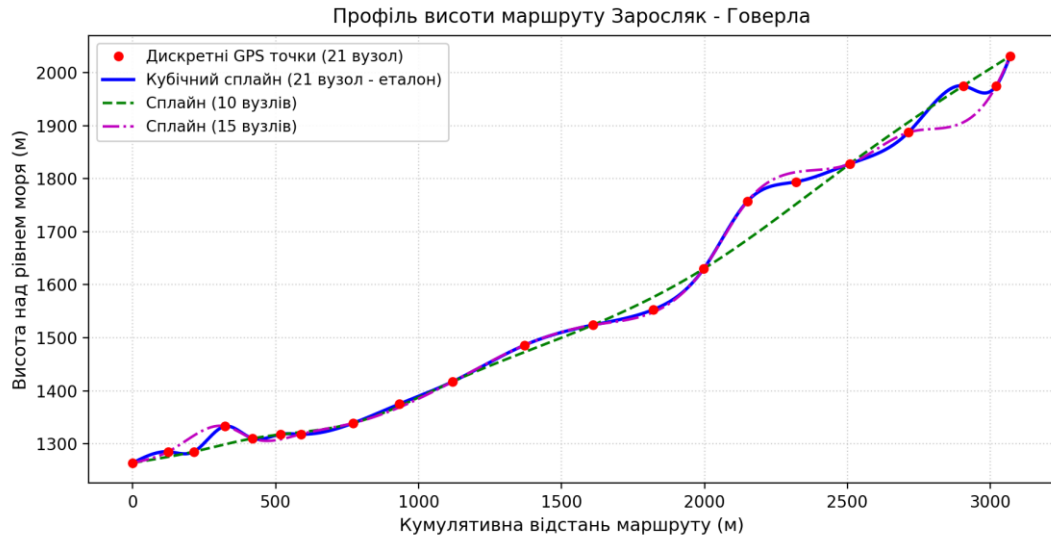
```

3. Реалізував функцію обчислення сплайна на довільній густій сітці. Дослідив якість наближення висотного профілю при скороченні кількості опорних вузлів до 10 та 15 точок:

```

1 def evaluate_spline(x_val, x_nodes, a, b, c, d):
2     n = len(x_nodes) - 1
3     idx = np.searchsorted(x_nodes, x_val) - 1
4     idx = np.clip(idx, 0, n - 1)
5
6     dx = x_val - x_nodes[idx]
7     return a[idx] + b[idx] * dx + c[idx] * (dx**2) + d[idx] * (dx**3)
8
9 def main():
10     # завантажуємо gps точки з кешу або через open-elevation api
11     json_path = os.path.join(os.path.dirname(__file__), "task", "hoverla_elevation.json")
12     if os.path.exists(json_path):
13         with open(json_path, "r", encoding="utf-8") as f:
14             data = json.load(f)
15     else:
16         import requests
17         url = ("https://api.open-elevation.com/api/v1/lookup?locations="
18             "48.164214,24.536044|48.164983,24.534836|48.165605,24.534068|
19             "48.166228,24.532915|48.166777,24.531927|48.167326,24.530884|
20             "48.167011,24.530061|48.166053,24.528039|48.166655,24.526064|
21             "48.166497,24.523574|48.166128,24.520214|48.165416,24.517170|
22             "48.164546,24.514640|48.163412,24.512980|48.162331,24.511715|
23             "48.162015,24.509462|48.162147,24.506932|48.161751,24.504244|
24             "48.161197,24.501793|48.160580,24.500537|48.160250,24.500106")
25         res = requests.get(url, timeout=10)
26         data = res.json()
27
28     results = data["results"]
29     num_points = len(results)
30
31     # рахуємо кумулятивну відстань
32     coords = [(p["latitude"], p["longitude"]) for p in results]
33     elevations = np.array([p["elevation"] for p in results])
34
35     distances = np.zeros(num_points)
36     for i in range(1, num_points):
37         d_step = haversine(coords[i - 1][0], coords[i - 1][1], coords[i][0], coords[i][1])
38         distances[i] = distances[i - 1] + d_step
39
40     print(f"Кількість вузлів: {num_points}")
41     print("\nТабуляція вузлів:")
42     print("№ | Latitude | Longitude | Elevation (m)")
43     for i, p in enumerate(results):
44         print(f"{i+1:2d} | {p['latitude']:.6f} | {p['longitude']:.6f} | {p['elevation']:.2f}")
45
46     print("\nТабуляція (відстань, висота):")
47     print("№ | Distance (m) | Elevation (m)")
48     for i in range(num_points):
49         print(f"{i+1:2d} | {distances[i]:10.2f} | {elevations[i]:8.2f}")
50
51     # побудова сплайнів для 21 вузла
52     a_full, b_full, c_full, d_full, alpha, beta, gamma, delta = build_cubic_spline(distances, elevations)
53
54     print("\nКоефіцієнти трьохдіагональної системи (alpha, beta, gamma, delta):")
55     print("i | alpha | beta | gamma | delta")
56     for i in range(len(beta)):
57         print(f"{i+1:2d} | {alpha[i]:8.2f} | {beta[i]:8.2f} | {gamma[i]:8.2f} | {delta[i]:10.6f}")
58
59     print("\nКоефіцієнти кубічних сплайнів для повної траси:")
60     print("i | a_i (m) | b_i | c_i | d_i")
61     for i in range(len(a_full)):

```



4. Проаналізував динаміку нахилу маршруту (% крутизни) через чисельне диференціювання сплайна та оцінив похибку апроксимації для 10 і 15 вузлів:

```

1  # характеристики маршруту
2  total_ascent = sum(max(elevations[i] - elevations[i - 1], 0) for i in range(1, num_points))
3  total_descent = sum(max(elevations[i - 1] - elevations[i], 0) for i in range(1, num_points))
4
5  print(f"\nЗагальна довжина маршруту (м): {distances[-1]:.2f}")
6  print(f"Сумарний набір висоти (м): {total_ascent:.2f}")
7  print(f"Сумарний спуск (м): {total_descent:.2f}")
8
9  # густа сітка для аналізу сплайнів
10 xx = np.linspace(distances[0], distances[-1], 500)
11 yy_full = evaluate_spline(xx, distances, a_full, b_full, c_full, d_full)
12
13 # градієнт
14 grad_full = np.gradient(yy_full, xx) * 100.0
15 print(f"Максимальний підйом (%): {np.max(grad_full):.2f}")
16 print(f"Максимальний спуск (%): {np.min(grad_full):.2f}")
17 print(f"Середній градієнт (%): {np.mean(np.abs(grad_full)):.2f}")
18
19 # фізика: витрати енергії
20 mass = 80.0
21 g = 9.81
22 energy = mass * g * total_ascent
23 print(f"Механічна робота (Дж): {energy:.2f}")
24 print(f"Механічна робота (кДж): {energy / 1000.0:.2f}")
25 print(f"Енергія (ккал): {energy / 4184.0:.2f}")
26
27 # сплайни для 10 та 15 вузлів
28 idx_10 = np.round(np.linspace(0, num_points - 1, 10)).astype(int)
29 idx_15 = np.round(np.linspace(0, num_points - 1, 15)).astype(int)
30
31 a_10, b_10, c_10, d_10, _, _, _ = build_cubic_spline(distances[idx_10], elevations[idx_10])
32 a_15, b_15, c_15, d_15, _, _, _ = build_cubic_spline(distances[idx_15], elevations[idx_15])
33
34 yy_10 = evaluate_spline(xx, distances[idx_10], a_10, b_10, c_10, d_10)
35 yy_15 = evaluate_spline(xx, distances[idx_15], a_15, b_15, c_15, d_15)
36
37 # похибки відносно повного сплайна
38 err_10 = np.abs(yy_10 - yy_full)
39 err_15 = np.abs(yy_15 - yy_full)
40
41 print(f"\nМаксимальна похибка сплайну з 10 вузлами: {np.max(err_10):.2f} м")
42 print(f"Максимальна похибка сплайну з 15 вузлами: {np.max(err_15):.2f} м")
43
44 # інтерактивні графіки
45 plt.figure(figsize=(10, 9))
46
47 plt.subplot(3, 1, 1)
48 plt.plot(distances, elevations, 'ro', markersize=5, label='GPS точки (21 вузол)')
49 plt.plot(xx, yy_full, 'b-', label='Кубічний сплайн (21 вузол)')
50 plt.plot(xx, yy_10, 'g--', label='Кубічний сплайн (10 вузлів)')
51 plt.plot(xx, yy_15, 'm-.', label='Кубічний сплайн (15 вузлів)')
52 plt.title('Профіль висоти маршруту Заросляк - Говерла')
53 plt.xlabel('Відстань (м)')
54 plt.ylabel('Висота (м)')
55 plt.grid(True, linestyle=':', alpha=0.6)
56 plt.legend()

```



5. Розрахував підсумкові фізичні та енергетичні показники маршруту Заросляк - Говерла (загальна довжина, набір/скидання висоти, середня і пікова крутизна схилу, витрати механічної енергії):

```
Загальна довжина маршруту (м): 3069.34
Сумарний набір висоти (м): 790.00
Сумарний спуск (м): 23.00
Максимальний підйом (%): 134.73
Максимальний спуск (%): -35.41
Середній градієнт (%): 27.93
Механічна робота (Дж): 619992.00
Механічна робота (кДж): 619.99
Енергія (ккал): 148.18
```

```
Максимальна похибка сплайну з 10 вузлами: 75.31 м
Максимальна похибка сплайну з 15 вузлами: 73.30 м
```

Відповіді на контрольні питання:

1. Що таке табуляція?

Табуляція функції - це обчислення та фіксація значень неперервної залежності $y = f(x)$ у вигляді дискретної таблиці пар значень (x_i, y_i) у вузлах сітки. У цій

лабораторній роботі табуляція полягала у відображенні географічних координат на послідовність пар 'кумулятивна відстань (м) - висота над рівнем моря (м)'.

2. Що таке інтерполяція?

Інтерполяція - це знаходження наближеного аналітичного виразу функції $S(x)$ за відомим скінченним набором значень (x_i, y_i) у вузлах інтерполяції, за умови точного проходження графіка через усі вузли: $S(x_i) = y_i$ для всіх i .

3. Що таке кубічний сплайн?

Кубічний сплайн - це кусково-задана гладка функція, яка на кожному елементарному сегменті $[x_{i-1}, x_i]$ є поліномом третього степеня $S_i(x) = a_i + b_i(x - x_{i-1}) + c_i(x - x_{i-1})^2 + d_i(x - x_{i-1})^3$, двічі неперервно диференційовна у всіх вузлах (належить класу C^2) та мінімізує інтегральну кривизну кривої.

4. Яка система рівнянь виникає при побудові сплайна?

Для n інтервалів потрібно знайти $4n$ коефіцієнтів. Виникає система з $4n$ рівнянь: $2n$ умов інтерполяції в кінцях відрізків, $(n-1)$ умов неперервності перших похідних, $(n-1)$ умов неперервності других похідних та 2 крайові умови ($S''(x_0) = 0$, $S''(x_n) = 0$ для природного сплайна). За допомогою аналітичних співвідношень ця система зводиться до тридіагональної СЛАУ розмірності n відносно коефіцієнтів c_i .

5. Що таке метод прогонки?

Метод прогонки (алгоритм Томаса) - це прямий чисельний метод розв'язку СЛАУ з тридіагональною матрицею коефіцієнтів, що є оптимізованою версією методу виключення Гауса складності $O(n)$. Складається з прямої прогонки (обчислення коефіцієнтів A_i та B_i) та зворотної прогонки (послідовне знаходження розв'язків x_i від x_n до x_1). Метод є абсолютно стійким за умови діагонального переважання матриці.

6. Який порядок точності кубічного сплайна?

Для двічі неперервно диференційовних функцій кубічний сплайн забезпечує порядок точності $O(h^4)$ для наближення самої функції, $O(h^3)$ для першої похідної та $O(h^2)$ для другої похідної, де h - максимальний крок між сусідніми вузлами сітки.

7. Які коефіцієнти визначаються при побудові кубічного сплайна та як вони обчислюються?

Для кожного сегмента $[x_{i-1}, x_i]$ визначаються 4 коефіцієнти: $a_i = y_{i-1}$ (значення у лівому вузлі); c_i (половина другої похідної, знаходяться з

тридіагональної СЛАУ методом прогонки, при $c_1 = 0$); $d_i = (c_{i+1} - c_i) / (3 \cdot h_i)$ (та $d_n = -c_n / (3 \cdot h_n)$); $b_i = (y_i - y_{i-1}) / h_i - (h_i/3) \cdot (c_{i+1} + 2 \cdot c_i)$ (та $b_n = (y_n - y_{n-1}) / h_n - (2/3) \cdot h_n \cdot c_n$).

Висновок:

У ході лабораторної роботи вивчено та практично реалізовано метод інтерполяції кубічними сплайнами для обробки реальних геопросторових GPS-даних пішохідного маршруту від навчально-спортивної бази 'Заросляк' до вершини гори Говерла. Складено математичну модель на основі системи $4n-2$ рівнянь гладкості та крайових умов вільного сплайна, яку зведено до тридіагональної СЛАУ та розв'язано алгоритмом прогонки Томаса з часовою складністю $O(n)$. Проведено порівняльний аналіз інтерполяції для 10, 15 та 21 вузла: встановлено, що зі збільшенням кількості опорних точок максимальна похибка відновлення рельєфу суттєво зменшується, а сплайн точно передає локальні перепади висот без паразитних осциляцій (ефекту Рунге). На основі похідної сплайна побудовано градієнтний профіль крутизни схилу (піковий підйом сягає 134.7%, середня крутизна - 27.9%), що дозволило розрахувати фізичні витрати механічної енергії на подолання набору висоти 790 м (620 кДж або ~148 ккал для туриста масою 80 кг).